A - Crear notebook y fijar semilla personal

Actividad 2.12 — Implementación de redes neuronales

Estudiante: Pohla López Número de lista: 22 Fecha:10/11/2026

En este laboratorio entrenaré un perceptrón multicapa para clasificar tumores benignos y malignos. Usaré MLflow para registrar los experimentos y comparar los resultados al cambiar el escalado, la arquitectura, la activación y la semilla.

Instalación, librerías y semilla

```
!pip install -q mlflow pyngrok
import os
import random
import subprocess
import time
from pathlib import Path

import kagglehub
import matplotlib.pyplot as plt
import mlflow
import numpy as np
import pandas as pd
import requests
import sklearn

from google.colab import userdata
from pyngrok import ngrok
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    f1_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

SEED = 22

random.seed(SEED)
np.random.seed(SEED)

print("Semilla:", SEED)
print("Pandas:", pd.__version__)
print("Scikit-learn:", sklearn.__version__)
print("MLflow:", mlflow.__version__)
```

```
Semilla: 22
Pandas: 2.2.3
Scikit-learn: 1.6.1
MLflow: 3.18.0
```

B - Carga y preparación de datos

Descargar el dataset

```
os.environ["KAGGLE_API_TOKEN"] = userdata.get("KAGGLE_API_TOKEN")

carpeta = Path(
    kagglehub.dataset_download("uciml/breast-cancer-wisconsin-data")
)

print("Archivos descargados:")

for archivo in carpeta.rglob("*"):
    if archivo.is_file():
        print(archivo.name)
```

```
Using Colab cache for faster access to the 'breast-cancer-wisconsin-data' dataset.
Archivos descargados:
data.csv
```

Cargar y revisar

```
datos = pd.read_csv(carpeta / "data.csv")

print("Dimensiones:", datos.shape)
datos.info()

display(datos.isna().sum().to_frame("nulos"))
```

```
Dimensiones: (569, 33)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   id                       569 non-null    int64  
 1   diagnosis                569 non-null    object 
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

	nulos
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0
Unnamed: 32	569

Preparar los datos

```
datos = datos.drop(columns=["id", "Unnamed: 32"])

datos["diagnosis"] = datos["diagnosis"].map({
    "M": 1,
    "B": 0,
})

print("Dimensiones después de eliminar las columnas:", datos.shape)

display(datos["diagnosis"].value_counts().to_frame("cantidad"))
```

```
Dimensiones después de eliminar las columnas: (569, 31)
```

diagnosis	cantidad
0	357
1	212

Balance de clases

El dataset contiene 357 casos benignos y 212 casos malignos. Hay más registros benignos, por lo que las clases no tienen la misma cantidad de ejemplos.

La etiqueta diagnosis quedó codificada con 0 para benigno y 1 para maligno.

Comparar las escalas

```
columnas = ["radius_mean", "area_mean", "smoothness_mean"]

escalas = datos[columnas].agg(["min", "max"]).T
escalas["rango"] = escalas["max"] - escalas["min"]

display(escalas)

relacion = (
    escalas.loc["area_mean", "rango"]
    / escalas.loc["smoothness_mean", "rango"]
)

print("El rango de area_mean es", round(relacion, 2),
      "veces el rango de smoothness_mean.")
```

	min	max	rango
radius_mean	6.98100	28.1100	21.12900
area_mean	143.50000	2501.0000	2357.50000
smoothness_mean	0.05263	0.1634	0.11077

```
El rango de area_mean es 21282.84 veces el rango de smoothness_mean.
```

Pasos siguientes: Generar código con escalas | New interactive sheet

Escala de las variables

El rango de area_mean es 21282.84 veces el de smoothness_mean. Esto muestra que las mediciones tienen magnitudes muy diferentes, aunque todas se utilizan como entradas del modelo.

C - Partición de los datos

Separar entrenamiento y prueba

```
X = datos.drop(columns="diagnosis")
y = datos["diagnosis"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=SEED,
)

print("Entrenamiento:", X_train.shape)
print("Prueba:", X_test.shape)

proporciones = pd.DataFrame({
    "entrenamiento": y_train.value_counts(normalize=True),
    "prueba": y_test.value_counts(normalize=True),
})

display(proporciones.sort_index())
```

```
Entrenamiento: (455, 30)
Prueba: (114, 30)
```

diagnosis	entrenamiento	prueba
0	0.626374	0.631579
1	0.373626	0.368421

Partición de los datos

Se utilizó el 80 % de los registros para entrenamiento y el 20 % para prueba. La partición fue estratificada para mantener una proporción similar de casos benignos y malignos en ambos conjuntos.

Esta partición se mantendrá fija durante todos los experimentos.

D - Desplegar MLflow en Colab

Iniciar MLflow

```
import os, subprocess, time, requests
from google.colab import userdata
from pyngrok import ngrok
from pyngrok.exception import PyngrokNgrokError
```

```
# 1) Servidor de seguimiento (tracking server) con SQLite como backend store.
#    Las dos variables permiten abrir la UI desde la URL pública de ngrok:
#    MLflow >= 3.5 rechaza por defecto cualquier Host distinto de localhost.
env = dict(os.environ,
           MLFLOW_SERVER_ALLOWED_HOSTS="*",
           MLFLOW_SERVER_CORS_ALLOWED_ORIGINS="*")
subprocess.Popen(["mlflow", "server",
                  "--backend-store-uri", "sqlite:///mlflow.db",
                  "--host", "127.0.0.1", "--port", "5000"], env=env)

# 2) Esperar a que el servidor responda
for _ in range(60):
    try:
        if requests.get("http://127.0.0.1:5000/health", timeout=1).ok:
            break
    except requests.ConnectionError:
        pass
    time.sleep(1)

# 3) Túnel público con ngrok
try:
    ngrok.set_auth_token(userdata.get("NGROK_AUTHTOKEN"))
except PyngrokNgrokError as e:
    if "already set" not in str(e):
        raise e

ngrok.kill()                    # cierra túneles anteriores
tunel = ngrok.connect(5000)
print("MLflow UI:", tunel.public_url)
```

```
MLflow UI: https://margarita-explore-ferris.ngrok-free.dev
```

Conectar el notebook y crear el experimento

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("IEI097-L2.12-Lopez_Moya")
```

```
2026/11/11 02:44:26 INFO mlflow.tracking.fluent: Experiment with name 'IEI097-L2.12-Lopez_Moya' does not exist. Creating a new experiment.
<Experiment: artifact_location='mlflow-artifacts:/1', creation_time=1794365067004, effective_trace_archival_retention=None, experiment_id='1',
last_update_time=1794365067004, lifecycle_stage='active', name='IEI097-L2.12-Lopez_Moya', tags={}, trace_location=None, workspace='default'>
```

E - Experimentos controlados

Creo que el run B tendrá mejores resultados que el run A porque utilizará las variables escaladas. Espero que esto ayude al modelo a aprender y detectar más casos malignos.

Función para ejecutar y registrar los experimentos

```
def correr_experimento(nombre, escalado, capas, activacion, semilla, fase):
    random.seed(semilla)
    np.random.seed(semilla)

    modelo = MLPClassifier(
        hidden_layer_sizes=capas,
        activation=activacion,
        alpha=0.0001,
        max_iter=500,
        random_state=semilla,
    )

    pipeline = Pipeline([
        ("escalado", StandardScaler() if escalado else "passthrough"),
        ("modelo", modelo),
    ])

    with mlflow.start_run(run_name=nombre):
        mlflow.set_tag("fase", fase)

        mlflow.log_params({
            "arquitectura": str(capas),
            "activacion": activacion,
            "alpha": modelo.alpha,
            "max_iter": modelo.max_iter,
            "random_state": semilla,
            "escalado": escalado,
            "semilla_particion": SEED,
        })

        pipeline.fit(X_train, y_train)

        pred_train = pipeline.predict(X_train)
        pred_test = pipeline.predict(X_test)

        metricas = {
            "accuracy_train": accuracy_score(y_train, pred_train),
            "accuracy_test": accuracy_score(y_test, pred_test),
            "recall_maligno": recall_score(y_test, pred_test),
            "f1": f1_score(y_test, pred_test),
            "n_iter": modelo.n_iter_,
        }

        mlflow.log_metrics(metricas)

        for epoca, perdida in enumerate(modelo.loss_curve_, start=1):
            mlflow.log_metric("train_loss", perdida, step=epoca)

        matriz = ConfusionMatrixDisplay.from_predictions(
            y_test,
            pred_test,
            labels=[0, 1],
            display_labels=["Benigno", "Maligno"],
        )
```

```
        matriz.ax_.set_title(nombre)
        mlflow.log_figure(matriz.figure_, "matriz_confusion.png")
        plt.close(matriz.figure_)

    return metricas
```

Run A — Sin escalado

```
resultado_a = correr_experimento(
    "A_sin_escalado",
    False,
    (30,),
    "relu",
    SEED,
    "experimentos",
)

display(pd.Series(resultado_a))
```

```
🏃 View run A_sin_escalado at: http://127.0.0.1:5000/#/experiments/1/runs/76a49c1773a742bf9467122dd7d71d3e
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	0
accuracy_train	0.940659
accuracy_test	0.956140
recall_maligno	0.880952
f1	0.936709
n_iter	208.000000

dtype: float64

El run A obtuvo una exactitud en prueba de 0.956140 y un recall maligno de 0.880952. Tomaré estos resultados como referencia para compararlos con el modelo que incorpora escalado.

Run B — Con escalado

```
resultado_b = correr_experimento(
    "B_con_escalado",
    True,
    (30,),
    "relu",
    SEED,
    "experimentos",
)

display(pd.Series(resultado_b))
```

```
🏃 View run B_con_escalado at: http://127.0.0.1:5000/#/experiments/1/runs/10e8ae2df6324824a43cf663db02524e
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	0
accuracy_train	0.991209
accuracy_test	0.982456
recall_maligno	0.976190
f1	0.976190
n_iter	355.000000

dtype: float64

Con el escalado, la exactitud en prueba pasó de 0.956140 a 0.982456 y el recall maligno de 0.880952 a 0.976190. Esto coincide con mi hipótesis porque mejoraron ambas métricas. El modelo pasó de detectar aproximadamente el 88,10 % de los casos malignos al 97,62 %, por lo que dejó menos casos malignos clasificados como benignos.

Run C — Cambio de arquitectura

```
resultado_c = correr_experimento(
    "C_dos_capas_relu",
    True,
    (64, 32),
    "relu",
    SEED,
    "experimentos",
)

display(pd.Series(resultado_c))
```

```
🏃 View run C_dos_capas_relu at: http://127.0.0.1:5000/#/experiments/1/runs/492db7425b484a2bb0370719c1d57862
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	0
accuracy_train	1.000000
accuracy_test	0.982456
recall_maligno	1.000000
f1	0.976744
n_iter	189.000000

dtype: float64

Al cambiar la arquitectura de (30,) a (64, 32), la exactitud en prueba se mantuvo en 0.982456 y el recall maligno aumentó de 0.976190 a 1.000000. Con esta semilla, el modelo detectó todos los casos malignos del conjunto de prueba, aunque la proporción total de aciertos se mantuvo igual.

Run D — Cambio de activación

```
resultado_d = correr_experimento(
    "D_dos_capas_tanh",
    True,
    (64, 32),
    "tanh",
    SEED,
    "experimentos",
)

display(pd.Series(resultado_d))
```

```
🏃 View run D_dos_capas_tanh at: http://127.0.0.1:5000/#/experiments/1/runs/11085c96aaa44d51ae8ffa3176ca9fa8
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	0
accuracy_train	1.000000
accuracy_test	0.964912
recall_maligno	0.976190
f1	0.953488
n_iter	307.000000

dtype: float64

Al cambiar relu por tanh, la exactitud en prueba pasó de 0.982456 a 0.964912 y el recall maligno de 1.000000 a 0.976190. En esta comparación, ambas métricas disminuyeron. Con la semilla 22, relu obtuvo mejores resultados y detectó todos los casos malignos del conjunto de prueba.

F - Semilla y replicación

Variabilidad entre cinco semillas

Repetiré la configuración C con las semillas 22, 23, 24, 25 y 26. Mantendré fija la partición de entrenamiento y prueba.

```
resultados_semillas = []

for semilla in range(SEED, SEED + 5):
    resultado = correr_experimento(
        f"C_semilla_{semilla}",
        True,
        (64, 32),
        "relu",
        semilla,
        "semillas",
    )

    resultados_semillas.append({
        "semilla": semilla,
        **resultado,
    })

tabla_semillas = pd.DataFrame(resultados_semillas)

display(tabla_semillas)
```

```
🏃 View run C_semilla_22 at: http://127.0.0.1:5000/#/experiments/1/runs/ec9c1fe20e1648c2b78c94bf566d53a6
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
🏃 View run C_semilla_23 at: http://127.0.0.1:5000/#/experiments/1/runs/eb8f9a04c2ba4aa891e11edddced66fc
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
🏃 View run C_semilla_24 at: http://127.0.0.1:5000/#/experiments/1/runs/fdfa29fbeb0740b4a2220689b2491228
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
🏃 View run C_semilla_25 at: http://127.0.0.1:5000/#/experiments/1/runs/22d62e28a0614c6d9a5b62a8220349d0
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
🏃 View run C_semilla_26 at: http://127.0.0.1:5000/#/experiments/1/runs/142f64ed76c1456c829ab0e043b3c4fd
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	semilla	accuracy_train	accuracy_test	recall_maligno	f1	n_iter
0	22	1.0	0.982456	1.00000	0.976744	189
1	23	1.0	0.982456	1.00000	0.976744	211
2	24	1.0	0.982456	1.00000	0.976744	186
3	25	1.0	0.982456	1.00000	0.976744	213
4	26	1.0	0.982456	0.97619	0.976190	170

Pasos siguientes: Generar código con tabla_semillas | New interactive sheet

Mínimo, máximo y rango

```
resumen = tabla_semillas[
    ["accuracy_test", "recall_maligno"]
].agg(["min", "max"]).T

resumen["rango"] = resumen["max"] - resumen["min"]

display(resumen)
```

	min	max	rango
accuracy_test	0.982456	0.982456	0.00000
recall_maligno	0.976190	1.000000	0.02381

Pasos siguientes: Generar código con resumen | New interactive sheet

Replicación del run C

Ejecutaré nuevamente la configuración C con la semilla 22 para comprobar si se repiten sus métricas.

```
resultado_replica = correr_experimento(
    f"C_replica_{SEED}",
    True,
    (64, 32),
    "relu",
    SEED,
    "replica",
)

comparacion = pd.DataFrame({
    "run_C": resultado_c,
    "replica": resultado_replica,
})

comparacion["diferencia_absoluta"] = (
    comparacion["run_C"] - comparacion["replica"]
).abs()

display(comparacion)
```

```
🏃 View run C_replica_22 at: http://127.0.0.1:5000/#/experiments/1/runs/68b9a8b0fbb64b828173da01773b525b
🧪 View experiment at: http://127.0.0.1:5000/#/experiments/1
```

	run_C	replica	diferencia_absoluta
accuracy_train	1.000000	1.000000	0.0
accuracy_test	0.982456	0.982456	0.0
recall_maligno	1.000000	1.000000	0.0
f1	0.976744	0.976744	0.0
n_iter	189.000000	189.000000	0.0

Pasos siguientes: Generar código con comparacion | New interactive sheet

Al comparar el run C con su réplica, las métricas coincidieron.

G - Analizar ejecuciones en MLflow y exportar evidencias

Obtener y exportar las ejecuciones

```
experimento = mlflow.get_experiment_by_name("IEI097-L2.12-Lopez_Moya")

runs = mlflow.search_runs(
    experiment_ids=[experimento.experiment_id]
)

columnas = [
    "run_id",
    "tags.mlflow.runName",
    "tags.fase",
    "params.arquitectura",
    "params.activacion",
    "params.alpha",
    "params.max_iter",
    "params.random_state",
    "params.escalado",
    "params.semilla_particion",
    "metrics.accuracy_train",
    "metrics.accuracy_test",
    "metrics.recall_maligno",
    "metrics.f1",
    "metrics.n_iter",
 ]

tabla_runs = runs[columnas]

display(tabla_runs)

tabla_runs.to_csv("runs.csv", index=False)
```

	run_id	tags.mlflow.runName	tags.fase	params.arquitectura	params.activacion	params.alpha	params.max_iter	params.random_state
0	68b9a8b0fbb64b828173da01773b525b	C_replica_22	replica	(64, 32)	relu	0.0001	500	2
1	142f64ed76c1456c829ab0e043b3c4fd	C_semilla_26	semillas	(64, 32)	relu	0.0001	500	2
2	22d62e28a0614c6d9a5b62a8220349d0	C_semilla_25	semillas	(64, 32)	relu	0.0001	500	2
3	fdfa29fbeb0740b4a2220689b2491228	C_semilla_24	semillas	(64, 32)	relu	0.0001	500	2
4	eb8f9a04c2ba4aa891e11edddced66fc	C_semilla_23	semillas	(64, 32)	relu	0.0001	500	2
5	ec9c1fe20e1648c2b78c94bf566d53a6	C_semilla_22	semillas	(64, 32)	relu	0.0001	500	2
6	11085c96aaa44d51ae8ffa3176ca9fa8	D_dos_capas_tanh	experimentos	(64, 32)	tanh	0.0001	500	2
7	492db7425b484a2bb0370719c1d57862	C_dos_capas_relu	experimentos	(64, 32)	relu	0.0001	500	2
8	10e8ae2df6324824a43cf663db02524e	B_con_escalado	experimentos	(30,)	relu	0.0001	500	2
9	76a49c1773a742bf9467122dd7d71d3e	A_sin_escalado	experimentos	(30,)	relu	0.0001	500	2

Pasos siguientes: Generar código con tabla_runs | New interactive sheet

Descargar runs.csv

```
from google.colab import files

files.download("runs.csv")
```

H - Cerrar y sincronizar el trabajo

1. ¿Qué cambió al escalar las variables?

En A_sin_escalado, la exactitud en prueba fue 0.956140 y el recall maligno fue 0.880952. En B_con_escalado, aumentaron a 0.982456 y 0.976190, respectivamente.

Esto coincide con mi hipótesis. Al aplicar escalado, las variables quedaron en magnitudes comparables y el modelo mejoró ambas métricas. La detección de casos malignos pasó aproximadamente del 88,10 % al 97,62 %.

2. ¿Más capas y neuronas mejoraron el resultado?

Al pasar de B a C, la exactitud en prueba se mantuvo en 0.982456 y el recall maligno aumentó de 0.976190 a 1.000000. Con la semilla 22, la arquitectura de dos capas detectó todos los casos malignos del conjunto de prueba.

Sin embargo, al repetir C con cinco semillas, el recall varió entre 0.976190 y 1.000000. Esa variación fue similar a la mejora entre B y C, por lo que estos resultados no bastan para afirmar que agregar más capas siempre mejora el modelo.

3. ¿Qué tan grande fue la variación entre semillas?

La exactitud en prueba se mantuvo en 0.982456 con las cinco semillas, por lo que su rango fue 0. El recall maligno tuvo un mínimo de 0.976190 y un máximo de 1.000000, con un rango de 0.02381, equivalente a aproximadamente 2,38 puntos porcentuales.

La variación del recall fue similar a la diferencia entre B y C. En cambio, fue menor que la mejora entre A y B, de aproximadamente 9,52 puntos porcentuales. Esto muestra que la semilla puede influir en los resultados y que conviene considerar varias ejecuciones antes de afirmar que una configuración es mejor.

4. ¿Qué error es más costoso y qué métrica priorizarías?

Considero más costoso el falso negativo, porque significa clasificar un tumor maligno como benigno y podría retrasar su detección. Un falso positivo clasifica un caso benigno como maligno y puede generar preocupación y evaluaciones adicionales.

Por eso priorizaría el recall de la clase maligna, que indica qué proporción de los casos realmente malignos detectó el modelo. En este laboratorio, el run C alcanzó un recall de 1.000000 en el conjunto de prueba.

5. ¿Qué información aporta MLflow y qué más registrarías?

MLflow permitió guardar los parámetros y las métricas de cada entrenamiento, comparar ejecuciones, revisar la pérdida por época y abrir las matrices de confusión. Con las impresiones que usamos en el notebook tendría que reunir esa información manualmente para compararla.

Cerrar el túnel

```
ngrok.kill()
```